

PRAKTIKUM SISTEM OPERASI

MODUL 6

(Proses Sekuensial dan Konkruen)



Oleh:

(Ammar Dzaki Nandana)

(2311104071)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

MODUL 6

1. Jumlah Maksimal Proses pada Xinu

Pada sistem operasi Xinu, jumlah maksimal proses dibatasi secara software melalui sebuah konstanta yang biasanya bernama:

```
#define NPROC 50
```

Lokasi file konfigurasi:

```
/include/conf.h
```

Secara default, nilai NPROC adalah:

```
#define NPROC 50
```

Artinya, Xinu hanya dapat membuat maksimal **50 proses** dalam satu waktu.

Mengubah Jumlah Maksimal Proses Menjadi 150

Langkah-langkah yang dilakukan:

1. Buka file konfigurasi:

```
vi /xinu/include/conf.h
```

1. Cari baris berikut:

```
#define NPROC 50
```

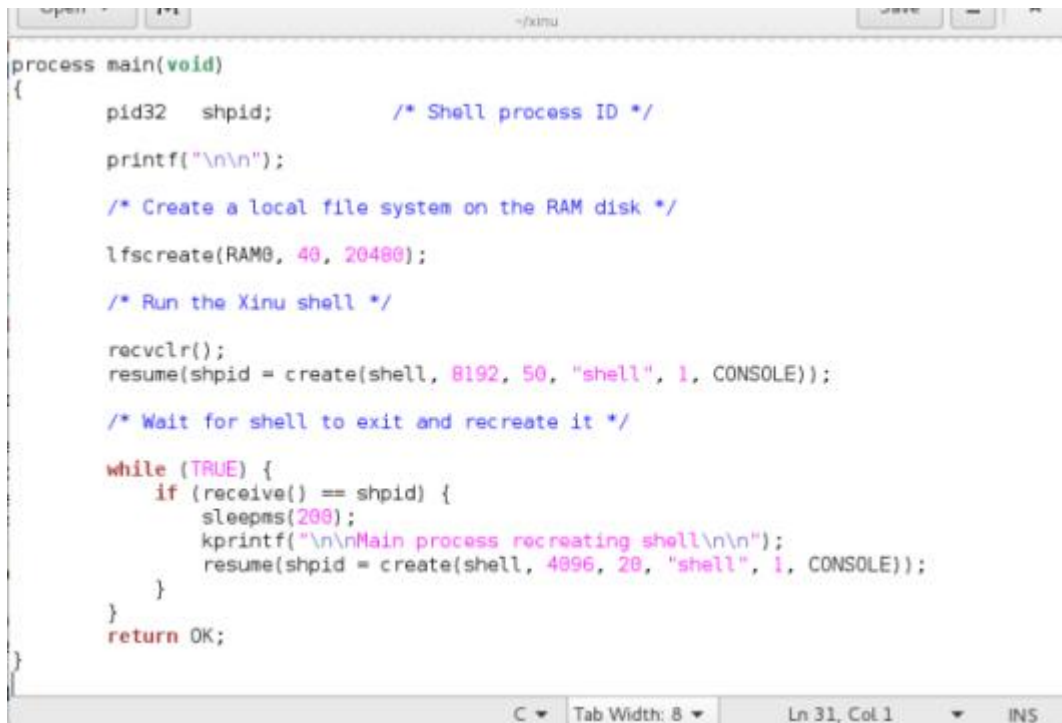
1. Ubah menjadi:

```
#define NPROC 150
```

1. Setelah melakukan perubahan, lakukan proses compile ulang Xinu:

```
make clean  
make
```

2. Menjalankan Kode main_ex1



```
process main(void)
{
    pid32  shpid;          /* Shell process ID */

    printf("\n\n");

    /* Create a local file system on the RAM disk */
    lfscreate(RAM0, 40, 20480);

    /* Run the Xinu shell */

    recvclr();
    resume(shpid = create(shell, 8192, 50, "shell", 1, CONSOLE));

    /* Wait for shell to exit and recreate it */

    while (TRUE) {
        if (receive() == shpid) {
            sleepms(200);
            kprintf("\n\nMain process recreating shell\n\n");
            resume(shpid = create(shell, 4096, 20, "shell", 1, CONSOLE));
        }
    }
    return OK;
}
```

Kode program Xinu pada percobaan ini merupakan contoh sederhana yang menunjukkan eksekusi fungsi secara berurutan tanpa menggunakan mekanisme multitasking maupun pembuatan proses baru.

Pada fungsi main(), terdapat pemanggilan dua fungsi, yaitu:

- sndA()
- sndB()

Namun, karena sndA() dipanggil terlebih dahulu dan di dalamnya terdapat perulangan tak hingga (while(1)), maka program akan terus menjalankan isi fungsi sndA() tanpa pernah kembali ke fungsi main() untuk menjalankan sndB().

Fungsi sndA() mencetak karakter "A" ke layar setiap 1 detik menggunakan:

```
printf();
sleepms(1000);
```

Sementara itu, fungsi sndB() memiliki struktur yang sama, yaitu mencetak karakter "B" setiap 1 detik. Akan tetapi, fungsi tersebut tidak pernah dieksekusi karena program sudah terjebak pada loop tak hingga di dalam sndA().

Akibatnya, output yang dihasilkan hanyalah karakter "A" yang muncul terus-menerus setiap detik.

Program ini menunjukkan bahwa tanpa pembuatan proses terpisah, misalnya menggunakan create() dan resume() pada Xinu, pemanggilan fungsi biasa tidak dapat berjalan secara paralel.

3. Menjalankan Kode main_ex2

Kode program pada percobaan ini menunjukkan implementasi multitasking dengan membuat dua proses yang berjalan secara bersamaan (*konkuren*).

Pada fungsi main(), digunakan fungsi create() untuk membuat dua proses baru, yaitu:

- sndA
- sndB

Parameter pada fungsi create() meliputi:

- Nama fungsi yang dijalankan
- Ukuran stack (1024)
- Prioritas proses (20)
- Nama proses (*string*)
- Jumlah argumen (0)

Setelah proses dibuat, fungsi resume() digunakan untuk mengaktifkan proses sehingga masuk ke dalam *ready queue* dan dapat dijadwalkan oleh scheduler Xinu.

Fungsi sndA() dan sndB() masing-masing memiliki perulangan tak hingga (while(1)) yang mencetak karakter "A" dan "B" ke layar setiap 1 detik menggunakan:

```
printf();  
sleepms(1000);
```

Karena kedua fungsi dijalankan sebagai proses yang berbeda, maka Xinu akan melakukan *context switching* di antara keduanya. Hal ini memungkinkan output "A" dan "B" muncul secara bergantian di layar, tergantung pada penjadwalan CPU oleh sistem operasi.

Berbeda dengan percobaan sebelumnya yang hanya menjalankan satu fungsi secara terus-menerus, pada percobaan ini kedua fungsi benar-benar berjalan secara paralel (*konkuren*), sehingga memperlihatkan konsep dasar manajemen proses pada sistem operasi Xinu.

4. Menjalankan Kode main_ex3

Pada percobaan ini, program Xinu menunjukkan penggunaan multitasking dengan parameter pada fungsi proses.

Di dalam fungsi main(), dibuat dua proses menggunakan create() yang menjalankan fungsi yang sama, yaitu sndChar, tetapi dengan argumen berbeda.

- Proses pertama menggunakan argumen karakter 'A' dengan nama proses "Send A".
- Proses kedua menggunakan argumen karakter 'B' dengan nama proses "Send B".

Parameter terakhir pada fungsi create(), yaitu 1, menunjukkan bahwa fungsi sndChar menerima satu argumen.

Setelah proses dibuat, keduanya diaktifkan menggunakan resume() sehingga dapat dijalankan oleh scheduler Xinu.

Fungsi:

```
sndChar(char c)
```

menerima parameter karakter c, kemudian menjalankan perulangan tak hingga (while(1)) yang mencetak karakter tersebut ke layar setiap 1 detik menggunakan:

```
printf();  
sleepms(1000);
```

Karena kedua proses berjalan secara konkuren, maka output yang dihasilkan berupa karakter "A" dan "B" yang muncul secara bergantian di layar sesuai penjadwalan CPU.

Program ini menunjukkan bahwa dalam Xinu, satu fungsi proses dapat digunakan kembali (*code reuse*) dengan parameter berbeda sehingga lebih efisien dibandingkan membuat dua fungsi terpisah.

Selain itu, program ini juga memperlihatkan bagaimana beberapa proses dapat berjalan secara paralel dan saling berbagi CPU melalui mekanisme penjadwalan.

5. Membuat Proses Produser dan Konsumer

Pada percobaan ini dibuat dua proses, yaitu:

- **Produser**
- **Konsumer**

Produser bertugas menghasilkan data berupa angka integer dari 1–1000.

Program menggunakan variabel global n yang diinisialisasi dengan nilai 0 dan digunakan sebagai data bersama.

Fungsi Produser

Fungsi produser() bertugas menghasilkan data dengan melakukan increment (n++) sebanyak 1000 kali melalui perulangan for.

Setelah proses selesai, nilai n akan menjadi 1000.

Fungsi Konsumer

Fungsi `konsumer()` bertugas menampilkan nilai `n` sebanyak 1000 kali menggunakan fungsi `printf()`.

Pada fungsi `main()`, pemanggilan dilakukan secara berurutan:

1. `produser()`
2. `konsumer()`

Karena dijalankan secara sekuensial, maka tidak terjadi *race condition* ataupun konflik akses data. Seluruh proses penambahan nilai pada `n` telah selesai sebelum dibaca oleh fungsi `konsumer`.

Akibatnya, output yang ditampilkan selalu menunjukkan nilai 1000 sebanyak 1000 kali.

Program ini menunjukkan bahwa tanpa penggunaan mekanisme proses paralel seperti `create()` dan `resume()`, konsep *producer-consumer* belum benar-benar terlihat karena belum terdapat eksekusi bersamaan maupun sinkronisasi antar proses.